

selinux总结

SELinux (Security-Enhanced Linux) 是由美国国家安全局 (NSA) 开发的种强制访问控制机制。它主要整合在 Linux 内核当中, 是针对特定的进程与指定的件资源进权限控制的系统。主要是增强传统 Linux 操作系统的安全性, 并解决传统 Linux 系统中主访问控制 (DAC) 系统中的各种权限问题 (如 root 权限过等)。

. DAC (主访问控制)

Linux 是个多多任务的分时操作系统, 如果要使系统资源, 就必须向系统管理员申请个账, 然后通过这个账进系统。这个账就是 ID。有时需要让多个具有相同的权限, 为了简化管理, 将具有相同特征组成个逻辑集合, 就叫组。件所属, 般为件的创建者, 谁创建了该件, 就天然的成为该件的所有者, 创建者所在组就是该件的所属组。
ls -l 显件的属性如下:

Shell

```
1 lrwxrwxrwx 1 root root    0 1970-01-01 08:00 eth0 ->
  .././devices/platform/soc/1c08000.qcom,pcie/pci0001:00/0001:00:00.0/0001:01:0
  0.0/net/eth0
2
3 -rw-r--r-- 1 root root 29431629 2009-01-01 08:00 fota.apk
4 drwxr-xr-x 3 root root    4096 2009-01-01 08:00 oat
```

前 10 个字符代表了标准 Linux 对 temp 这个件的访问控制, 后是件的所有者、所有者所在的组及其它属性:

- 第个字符代表件 (-)、录 (d), 链接 (l)
- 其余字符每 3 个组 (rwx), 读 (r)、写 (w)、执 (x)
- 第组 rwx: 件所有者的权限是读、写和执
- 第组 r-x: 与件所有者同组的的权限是可读不能写但是可以执
- 第三组 r-x: 不与件所有者同组的其他权限是可读不能写但是可以执

缺点:

- root 权限过, 使 rwx 权限对 root 并不效, 旦 root 被窃取或者 root 本的误操作, 就会对 Linux 系统产严重的安全问题。
- Linux 默认权限过于简单, 只有所有者、所属组和其他的份, 权限也只有读、写和执权 • 限, 并不利于权限细分与设定。

. MAC (强制访问控制)

MAC 是通过 SELinux 的默认策略规则来控制特定的进程对系统的件资源的访问。也就是说, 即使你是 root, 但是当你访问件资源时, 如果使了错误的进程, 那么也是不能访问的。在 SELinux 的强制访问控制下, 访问的主体不再是, 是进程。客体可以是件、录、共享内存、套接字、络主机等。主体和客体都被标记了固定的安全属性 (如安全等级、访问权限等), 在每次访问发时, 系统检测安全属性以便确定个主体是否有权访问该客体。

Android系统SELinux的两种作模式:

1. Permissive作模式 (宽容模式)
2. Enforcing作模式 (强制模式)

三. 基本概念

1. 主体 (subject)

基本等同于进程，所有进程的type必须归属于domain。domain因此成了进程type的常见表述。

2. 客体 (object)

被主体访问的资源：件（含可执行件），录，节点等，所有文件type都归属于file_type

3. 安全上下 (secure context)

3.1 查看安全上下法：

- ls -Z 查看件（如果是录则显其路径下的件）的安全上下

```
kona:/system/bin # ls -Z cat
u:object_r:system_file:s0 cat
```

- ps -eZ 查看进程的安全上下

```
parrot:/ # ps -eZ | grep bluetooth
u:r:hal_bluetooth_default:s0 bluetooth 1317 1 0 02:34:48 ? 00:00:00 android.hardware.bluetooth@1.0-service-qt...
```

- id -Z 查看shell的安全上下

```
parrot:/ # id -Z
u:r:su:s0
```

- getprop -Z 查看属性上下文

```
1|parrot:/ # getprop -Z ro.build.data
u:object_r:default_prop:s0
```

3.2. 安全上下格式

user : role : type : sensitivity[:categories]

user: , user是固定的，永远为u;

role: , role是固定的，subject永远为r，object永远为object_r。一般情况下，进程一方就是subject，文件一方就是object，所以一般进程的role为r，文件的role为object_r。

type: subject或者object所关联的标签，简单理解就是将一个或几个进程声明为Type A， 将一个或多个文件声明为Type B，那么在控制这个进程有没有权限访问这个文件的时候，只用A和B来表示就可以了。;

sensitivity: 是固定的，永远为s0，MLS将系统的进程和件进了分级，不同级别的资源需要对应级别的进程才能访问。

categories: 是Multi-Level Security (MLS) 协议，用来隔离一个app的数据，防止被另一个app访问，或者隔离不同用户间对同一个app data的访问。

3.3 进程Context (seapp_contexts)

给一个进程关联Context: user=_app isPrivApp=true name=com.android.vzwomatrigger domain=vzwomatrigger_app type=privapp_data_file levelFrom=all

user=_app代表这是一个常规的app

isPrivApp=true代表这是一个预置app

name=com.android.vzwomatrigger 指定了进程名

domain=vzwomatrigger_app 将进程名关联到一个type

type=privapp_data_file 这是一个文件type，指定了app的数据 directory目录所属的type

levelFrom=all MLS/MCS的level相关

AOSP内置进程Context见platform/system/sepolicy/private/seapp_contexts，供应商或厂商要定义自己的seapp_context，将在/vender下或<root>/device /manufacturer/device-name/sepolicy下相应目录添加。

3.4 文件Context (file_contexts)

给一个文件关联Context: `/system/bin/bcc`

`u:object_r:rs_exec:s0`

`/system/bin/bcc` 指的是具体文件

`u:object_r:rs_exec:s0` 是一个security context。其中的`rs_exec`为type。这样文件和type就进行了关联

3.5 Type

简单理解就是将一个或几个进程声明为Type A， 将一个或多个文件声明为Type B。那么在控制这个进程有没有权限访问这个文件的时候，只用A和B来表示就可以了。

“一类进程”总比具体的“一个进程”要灵活的多。把多个进程声明为同一个Type，那么在写规则的时候只要描述这个Type，那么这个Type对应的所有进程都会生效。文件或其他对象也是同样的。

也就是说，在规则中描述Source能不能访问Target，是通过Type来表述的。

Type是厂商或供应商可以自定义的，自定义type命令的完整格式为：`type type_id [alias alias_id,] [attribute_id]`

3.6 Attribute

将多个Type归为一组，就是一个Attribute。

通俗的说，把一些进程声明为Type，但是多个Type有某种共通的特性，就可以把这些Type声明为同一个Attribute。在描述规则的时候，可以将Source或者Target指定为一个Attribute而不是Type，这样所有属于这个Attribute的Type都生效。

AOSP本身内置了一些Attribute，而这些Attribute很多都是约定俗称的固定含义。比如：

`domain` 所有进程的type必须归属于`domain`。`domain`因此成了进程type的常见表述。

`file_type` 所有文件type都归属于`file_type`

AOSP内置的Attribute见：

`platform/system/sepolicy/public/attributes`

`platform/system/sepolicy/private/attributes`

`platform/system/sepolicy/prebuilts/api/[version]/public/attributes`

`platform/system/sepolicy/prebuilts/api/[version]/private/attributes`

4. 策略规则语句格式

ex : `allow source target : class permissions;`

Rule Name 上面规则示例中，`allow`就是Rule Name的一种。SELinux有多种RuleName：

`allow`：表示允许Source对Target执行许可的操作，`allow`是用的最多的。

`neverallow`：表示不允许Source对Target执行制定的操作

`auditallow`：表示允许操作并记录访问决策信息

`dontaudit`：表示不记录违反规则的决策信息，切违反规则不影响运行。

`source` 指的是访问者subject所关联的type或attribute。

`target` 指的时被访问object所关联的type或attribute。

`class` 指被访问者object的类型，可`system/sepolicy/private/security_classes`中的定义

file-related classes

class filesystem

class file #代表普通件

class dir #代表录

class fd #代表件描述符

```
class lnk_file #代表链接件
class chr_file #代表字符设备件

# network-related classes

class socket #socket

class tcp_socket

class udp_socket

class binder #Android 平台特有的 binder

class zygote #Android 平台特有的 zygote
```

permissions 表示具体可以做什么操作。不同的Class有不同的Permissions集合。Class与Perimssion的完整映射具体见源码：kernel/arm64/security/selinux/include/classmap.h。

Class	Permission
file	ioctl read write create getattr setattr lock relabelfrom relabelto append unlink link rename execute swapon quotaon mounto
directory	add_name remove_name reparent search rmdir open audit_access execmod
socket	ioctl read write create getattr setattr lock relabelfrom relabelto append bind connect listen accept getopt setopt shutdown re
filesystem	mount remount unmount getattr relabelfrom relabelto transition associate quotamod quotaget
process	fork transition sigchld sigkill sigstop signull signal ptrace getsched setsched getsession getpgid setpgid getcap setcap shar
security	compute_av compute_create compute_member check_context load_policy compute_relabel compute_user setenforce set
capability	chown dac_override dac_read_search fowner fsetid kill setgid setuid setpcap linux_immutable net_bind_service net_broad
MORE	AND MORE

6. SeLinux文件体系

6.1 TE文件

就是一堆.te文件。TE里定义了Type、访问规则等。对进程和文件Type的定义、访问规则的定制就只在TE里完成的。一般每个进程有一个独立的te文件，而所有文件的type统一声明在一个file.te文件中。

6.2 Context files

- file_contexts: file_contexts里面定义了文件的context，用于将文件与type关联起来。
- genfs_contexts: 用于将文件系统（比如/proc和vfat）与type关联起来。
- property_contexts: 用于将Android系统properties与type关联起来
- service_contexts: 用于将service进程与type关联
- seapp_contexts: seapp_context里除了可以将一个具体应用映射到一个domain，也可以将seinfo映射到domain。既未配置seinfo，又未配置seapp_context的app，将默认为untrusted_app。
- mac_permissions.xml: 根据app的signature或包名，为app分配一个seinfo。seinfo用于在app没有明确关联一个type时归属一个默认type。
- keystore2_key_contexts : 为Keystore 2.0 namespaces分配一个label。
- attribute: 用来声明Attribute。
- security_classes: 用来声明Class。
- te_macros: 定义了一些宏，可以简化一些权限的添加，如: binder_use(domain) 允许domain使用binder ipc通信
- global_macros: 定义了一些宏，可以简化权限语句的编写，如: allow system_app mnt_vendor_file:dir create_dir_perms; (allow system_app mnt_vendor_file:file {write open getattr create read setattr};)

6.3 Selinux目录

aosp目录

在Android 8以上的系统中，AOSP和厂商、供应商的部分是独立配置的，方便OTA更新。针对这种架构，SeLinux的Policy文件体系包含以下目录：

system/sepolicy/public: 这部分是AOSP公开给vender使用的，作为其基础api。比如声明了domain的attribute就在这下面：system/sepolicy/public/attributes。

system/sepolicy/private: AOSP内部使用的，即system image内部用的policy。vender不应该感知到这部分policy。

system/sepolicy/vendor: vender部分的policy。

device/manufacture/sepolicy: 第三方扩展的policy，比如高通设备：device/qcom/sepolicy。

高通目录

高通的主目录分为两个，一个是QSSI， 一个是VENDOR， QSSI下存放着Android原生的一些东西， VENDOR下放着厂商定制的东西。Selinux策略sepolicy在QSSI和VENDOR下都有，sepolicy分为原生sepolicy 和厂商自己定制的sepolicy。

QSSI/system/sepolicy: 这是原生的sepolicy，不要在原生的sepolicy目录下修改文件，否则会导致CTS失败。

根目录	子目录	编译后的生成路径	如果在这个目录下定义一个新的type， system/vendor访问限制
QSSI/system/sepolicy	private	out/target/product/qssi/system/etc/selinux	system可以访问， vendor无法访问
	public	out/target/product/qssi/system/etc/selinux	system， vendor都可访问
	private/compat	/	当在system/sepolicy/public新增一个type的时候，需要在compat/api/api.ignore.cil中添加这个新增的标签, 否则会编译失败

QSSI侧Sepolicy定制目录，在mk文件中使用SYSTEM_EXT_PUBLIC_SEPOLICY_DIRS、SYSTEM_EXT_PRIVATE_SEPOLICY_DIRS、PRODUCT_PUBLIC_SEPOLICY_DIRS、PRODUCT_PRIVATE_SEPOLICY_DIRS变量添加system_ext和product sepolicy扩展目录。

根目录	子目录	编译后的生成路径	如果在这个目录下定义一个新的type， system/vendor访问限制
QSSI/device/qcom/sepolicy/generic/	private	out/target/product/qssi/system_ext/etc/selinux	system可以访问， vendor无法访问
	public	out/target/product/qssi/system_ext/etc/selinux	system， vendor都可访问
	product/private	out/target/product/qssi/product/etc/selinux	system可以访问， vendor无法访问
	product/public	out/target/product/qssi/product/etc/selinux	system， vendor都可访问
	private/compat	/	当在device/qcom/sepolicy/generic/public新增一个type的时候，需要在compat/api/api.ignore.cil中添加这个新增的标签, 否则会编译失败

VENDOR侧Sepolicy定制目录，在mk中使用BOARD_SEPOLICY_DIRS变量添加vendor sepolicy扩展目录：

根目录	子目录	编译后的生成路径	如果在这个目录下定义一个新的type， system/vendor访问限制
VENDOR/device/qcom/sepolicy_vndr/generic/vendor/	common所有芯片共用的	out/target/product/qssi/vendor/etc/selinux	vendor可以访问， system无法访问
	还有其他子目录， 分别是不同的芯片不同的sepolicy		

四. SELinux问题的解法

1. 临时调试

adb shell getenforce 可查看设备selinux模式

adb shell setenforce 0 可设置设备selinux模式（0: permissive; 1: enforcing）

chcon [...] 选项功能: -u 修改安全上下文的用户字段 -r 修改安全上下文的角色字段 -t 修改安全上下文的类型字段 -l 修改安全上下文的级别字段 --reference 修改与指定文件或目录相一致的安全上下文 -R递归操作 -h修改软链接的安全上下文（不加此选项则修改软链接对应文件）

restorecon [选项] [...] 选项功能: -v打印操作过程 -R递归操作

chmod -R 666 file 递归的修改文件dac权限为可读写

2. 默认关闭SELinux

修改QSSI.14/system/core/init/selinux.cpp件

```
+10↑ - Show 739 common lines - +10↓

    }

    if (!android::base::ReadFdToString(policy_file.fd, policy)) {
        PLOG(FATAL) << "Failed to read policy file: " << policy_file.path;
    }
}

void SelinuxSetEnforcement() {
    bool kernel_enforcing = (security_getenforce() == 1);
    bool is_enforcing = IsEnforcing();
    is_enforcing = false;
    if (kernel_enforcing != is_enforcing) {
        if (security_setenforce(is_enforcing)) {
            PLOG(FATAL) << "security_setenforce(" << (is_enforcing ? "true" : "false")
                << ") failed";
        }
    }

    if (auto result = WriteFile("/sys/fs/selinux/checkreqprot", "0"); !result.ok()) {
        LOG(FATAL) << "Unable to write to /sys/fs/selinux/checkreqprot: " << result.error();
    }
}
```

3. 动添加规则

般权限问题都会在log件中打印，可通过过滤avc关键字查看，如下：

```
auditd : type=1400 audit(0.0:1275): avc: denied { open } for comm="monitor-thread"
path="/sys/devices/platform/soc/1c08000.qcom,pcie/pci0001:00/0001:00:00.0/0001:01:00.0
/net/eth0/operstate" dev="sysfs" ino=63866 scontext=u:r:system_app:s0
tcontext=u:object_r:sysfs:s0 tclass=file permissive=1
```

分析过程：

哪个进程缺少权限：monitor-thread

缺少什么权限： { open} 权限，

谁缺少权限： u:r:system_app:s0

对哪个件缺少权限： u:object_r:sysfs:s0

什么类型的件： file

完整的意思： monitor-thread(system_app)进程对sysfs类型的file缺少open权限。

接下来只需要在system_app.te件中加：

allow system_app sysfs:file open即可（全编译会报错，因为这句代码是给所有system_app进程开放了所有件的open权限）

4. 使audit2allow具添加规则

在ubuntu下执：adb shell dmesg |grep avc > avc; audit2allow -i avc, 例：

```
#===== system_app =====
allow system_app debugfs:file read;
allow system_app ethernet:binder call;
allow system_app self:udp_socket ioctl;
allow system_app sysfs:file { read getattr open };
allow system_app system_app_data_file:sock_file create;
```

直接把上述生成的让内容复制粘贴到对应的te件中（如上图应该将规则粘贴于system_app.te件中）

有时候avc denied的log不是次性暴露所有权限问题，要等解决个权限问题之后，才会暴露另外个权限问题。

如提缺少某个读的read权限，加read之后，才显缺少write权限，要次次次试，次次加，时间成本极。

针对dir缺少的任何权限，建议赋予create_dir_perms，基本涵盖对dir的所有权限，如：{ open search write read rename create rmdir getattr }等等。

针对file缺少的任何权限，建议赋予rw_x_file_perms，基本涵盖对file的所有权限，如：包含{ open read write open execute getattr create ioctl }等等。

5. 预置三商源码

第三方源码一般都会新建te件把selinux权限配好，预置时只需在mk件中引使用BOARD_SEPOLICY_DIRS += device/\$SoC/common/sepolicy 命令添加家定义的sepolicy 规则；

6. neverallow问题

般在项上遇到的最多的就是给节点加权限，这类情况使audit2allow具般就会给你句：allow xxx sysfs:file {read getattr open}，然后贴到对应的te件中，全编报错：neverallow xxxvim

因为sysfs:file 相当于我们开放了所有件的权限，这是不被允许的，所以我们只能对节点单独添加规则（如给/sys/class/net/eth0/operate添加权限）：

1. 在设备内部执ls -l /sys/class/net/eth0/ 发现它是个软链接（给软链接添加规则是效的），具体路径是：

```
/sys/devices/platform/soc/1c08000.qcom,pcie/pci0001:00/0001:00:00.0/0001:01:00.0/net/eth0/operstate
```

2. 在file_contexts中加上新的context，如下（sysfs_eth是我定义的类型）：

```
/sys/devices/platform/soc/1c08000.qcom,pcie/pci0001:00/0001:00:00.0/0001:01:00.0/net/eth0/operstate u:object_r:sysfs_eth:s0
```

3. 在file.te中加上：

```
type sysfs_eth, fs_type, sysfs_type;
```

4. 最后去system_app.te件中添加规则：

```
allow system_app sysfs_eth:file { read getattr open };
```

```
allow system_app sysfs_eth:dir search;
```

7. mls规则问题

如果te文件已经添加SELinux权限，但没有生效，查看AVC log信息出现“s0:c512,c768”字样，则可判断是由于mls规则导致。

说明主体和客体安全级别不同，可通过使主体进程关联mltrustedsubject或客体关联mltrustedobject来解决。

但是如果主客体之间的操作是connectto，这种情况与上种情况略有不同，主体或者客体都要关联mltrustedsubject。

问题现象：已经在platform_app.te中添加了SELinux权限，但log中依然有如下报错：

```
avc: denied { connectto } for pid=2002 comm=“slogHandlerThre” path=00736C6F676D6F64656D scontext=u:r:platform_app:s0:c512,c768
tcontext=u:r:slogmodem:s0 tclass=unix_stream_socket permissive=0
```

问题分析：这是因为谷歌在文件system/sepolicy/private/mls中进行了安全级的限制，限制代码如下：

```
#Stream connect: Client must be equivalent to server unless one of them is trusted.mlsconstrain unix_stream_socket { connectto }(l1 eq
l2 or t1 == mltrustedsubject or t2 == mltrustedsubject);
```

解决方法：这种情况需客体标签关联mltrustedsubject，这里platform_app最好不要修改，所以要修改客体slogmodem。具体修改方法如下：

打开device/sprd/mpool/module/log/msepolicy/system/public/slogmodem.te文件，添加如下代码：type slogmodem, domain, mltrustedsubject;

8. rc中新增service无法启动

问题现象：添加service，push到终端验证，service无法正常启动。开机失败，log如下：

```
00EF4 <11> [ 4.994631][03-08 22:21:38.994] init: Could not start service 'vendor.sprd.hardware.vdsp@1.0-service' as part of class 'hal':  
File /vendor/bin/hw/vendor.sprd.hardware.vdsp@1.0-service(labeled "u:object_r:vendor_file:s0") has incorrect label or no domain  
transition from u:r:init:s0 to another SELinux domain defined.
```

问题分析：正常编译的时候会设置标签，标签正确不会出现以上报错。采用push的场景需要手动执行恢复标签的命令。

解决方法：需要执行下命令恢复SELinux标签：adb shell restorecon -R /vendor/bin/hw/vendor.sprd.hardware.vdsp@1.0-service;